UNIVERSIDAD EVANGÉLICA DE EL SALVADOR

Facultad de Ingeniería

Ingeniería en Desarrollo de Software y Ciencia de Datos

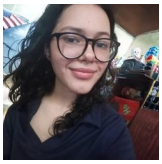
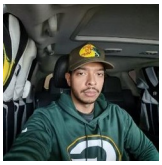
Programación II

Entrega final — Sistema para Administración de Asociación Comunal

Grupo 11

Facilitador:

Ing. Daniel Enrique Guevara Gómez

Foto	Nombre completo	CIF	¿Participó?
	Isabel Uvaldina Ramos Saravia	2026010478	SI
	Camila Nicole Salvador Samayoa	2026011056	SI
	Diego Emanuel Escobar Hernández	2026011594	SI
	Perla Esmeralda López Barrientos	2026011595	SI
	Luis Alonso Alvarenga Meza	2026010120	SI

San Salvador, El Salvador, octubre de 2026

2. Introducción

Este documento presenta la entrega final del proyecto de Programación II: un sistema para la administración de una Asociación Comunal desarrollado en Java 17, construido sobre los dos avances anteriores. El Avance 1 definió el problema, los roles, las historias de usuario y las entidades; el Avance 2 implementó el modelo orientado a objetos (herencia, clases abstractas, polimorfismo), la arquitectura por capas, el proceso concurrente de cierre mensual y la persistencia en archivos .dat. Esta entrega sustituye los archivos por una base de datos MySQL accedida mediante JDBC con un pool de conexiones (DataSource), incorpora un manejo de excepciones propio, pruebas unitarias con JUnit 5 y Mockito, y una revisión del código orientada a la cohesión, el bajo acoplamiento y la legibilidad.

El equipo se reorganizó a mediados del ciclo y quedó conformado por cinco integrantes bajo el nombre Grupo 11; el proyecto conserva íntegramente el trabajo de los avances previos.

2.1 Compromisos del Avance 2 y su cumplimiento

Las observaciones y compromisos registrados en el Avance 2 se atendieron de la siguiente forma:

Compromiso / observación	Estado	Dónde
Sustituir la persistencia en archivos .dat por JDBC	Cumplido	dao/JdbcSupport, SocioDAO, CuotaDAO, UsuarioDAO sobre MySQL con PreparedStatement, transacciones y pool de conexiones
Cifrado de contraseñas (se almacenaban en texto plano)	Cumplido	util/PasswordUtil: hash PBKDF2-HMAC-SHA256 con salt; Usuario.validarContrasena() compara contra el hash. Es hash (irreversible), más adecuado que cifrado para contraseñas
Interfaz web o de escritorio sobre los mismos servicios	Justificado (no realizado)	La rúbrica final no evalúa la interfaz y sí evalúa JDBC, pool, excepciones y pruebas (3.6 pts). Con cinco integrantes se priorizó lo evaluado; la arquitectura por capas deja la presentación intercambiable
Un solo usuario a la vez	Mantenido	Aplicación de consola; el pool y los DAO sin estado ya soportan varios hilos
Objetivo específico N°3	Cumplido	Sección 4

3. Descripción del problema

Las asociaciones comunales de El Salvador administran su padrón de socios, el cobro de cuotas mensuales y la conformación de su junta directiva, en la mayoría de los casos con cuadernos u hojas de cálculo. Esto produce padrones desactualizados, cuotas cobradas dos veces o nunca, dificultad para saber quién está en mora y poca transparencia hacia los socios sobre el manejo de los fondos.

El sistema resuelve esta problemática con un registro único de socios (con su estado de membresía y, cuando corresponde, su cargo en la directiva), la generación mensual de cuotas con un monto que depende del tipo de socio, el registro de pagos, el reporte de morosos y el acceso por roles (Administrador, Secretario, Tesorero y Socio), todo respaldado por una base de datos relacional que garantiza la integridad de la información.

4. Objetivos

4.1 Objetivo general

Desarrollar un sistema informático para la administración de una Asociación Comunal, que permita gestionar de manera eficiente el registro de socios, el control de sus cuotas y pagos, y la supervisión de la junta directiva, facilitando así la organización interna y la transparencia en el manejo de la información de la comunidad.

4.2 Objetivos específicos

- **N°1 (Avance 1):** Diseñar e implementar un módulo de registro y gestión de socios que permita almacenar su información personal y su estado de membresía.
- **N°2 (Avance 2):** Implementar los módulos de socios, junta directiva y cuotas aplicando POO, arquitectura por capas, persistencia y un cierre mensual concurrente.
- **N°3 (Entrega final):** Migrar la persistencia a MySQL mediante JDBC con pool de conexiones, integrar el proceso concurrente con la base de datos, incorporar manejo de excepciones propio y pruebas unitarias automatizadas, y revisar el código para elevar su cohesión, reducir su acoplamiento y mejorar su legibilidad.

5. Requerimientos

5.1 Roles y funciones del sistema

Rol	Funciones en el sistema
Administrador	Todas las funciones de Secretario y Tesorero; crear, listar y eliminar usuarios; cambiar estado de membresía; consultar el estado del pool de conexiones
Secretario	Registrar socios y miembros de la junta directiva; actualizar información; buscar por DUI; listar el padrón
Tesorero	Registrar pagos; consultar cuotas pendientes e historial; reporte de morosos; ejecutar el cierre mensual
Socio	Consultar sus datos, sus cuotas pendientes y su historial

5.2 Historias de usuario

ID	Prioridad	Historia	Criterios de aceptación
HU-001	Alta	Como Secretario, quiero registrar un nuevo socio para mantener actualizado el padrón.	DUI (########-#) y teléfono (####-####) validados; DUI único (validado en servicio y por UNIQUE en la BD); id asignado por la BD.
HU-002	Alta	Como Secretario, quiero actualizar la información de un socio.	Cambios de teléfono y dirección persistidos con UPDATE.
HU-003	Alta	Como Tesorero, quiero registrar el pago de una cuota.	Se listan las pendientes; la cuota queda PAGADA con fecha; pagar dos veces es rechazado.

ID	Prioridad	Historia	Criterios de aceptación
HU-004	Media	Como Tesorero, quiero consultar las cuotas pendientes de un socio.	Muestra PENDIENTES y VENCIDAS con el total adeudado.
HU-005	Media	Como Secretario, quiero el reporte de socios.	Padrón completo identificando a la directiva.
HU-006	Media	Como Tesorero, quiero el reporte de morosos.	Socios con cuotas vencidas, cantidad y monto, ordenados por monto.
HU-007	Baja	Como Socio, quiero consultar mi historial de cuotas.	Pagos y pendientes ordenados por periodo.
HU-008	Alta	Como usuario, quiero iniciar sesión para acceder solo a las funciones de mi rol.	Credenciales validadas contra el hash; menú según rol.
HU-009	Media	Como Secretario, quiero registrar miembros de la junta directiva con cargo y periodo.	Aparecen en el padrón con su cargo; cuota diferenciada ($7.50).
HU-010	Alta	Como Tesorero, quiero ejecutar el cierre mensual sin detener mi trabajo.	Genera una cuota por socio activo sin duplicar; marca vencidas; corre en segundo plano; resumen al consultar; errores de BD reportados.
HU-011	Baja	Como Administrador, quiero crear usuarios con rol.	Usuario único; contraseña mínima de 6 caracteres almacenada como hash; rol SOCIO ligado a un socio.
HU-012	Baja	Como Administrador, quiero eliminar usuarios que ya no operan el sistema.	DELETE del usuario; no puede eliminarse al único administrador.

6. Arquitectura

El sistema mantiene la arquitectura por capas del Avance 2 y agrega dos capas nuevas: repository (interfaces que abstraen el acceso a datos) y config (DataSource y pool). Cada capa conoce únicamente a la inmediata inferior y los servicios dependen de abstracciones, no de JDBC:

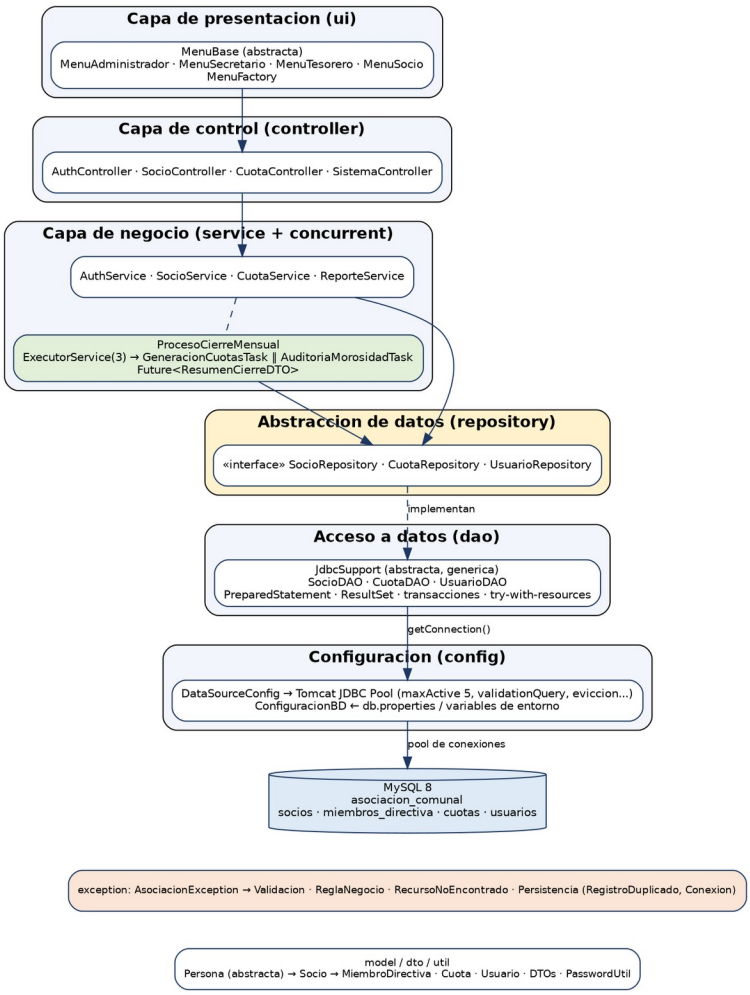


Figura 1. Arquitectura por capas y flujo de una operación hasta MySQL.

```
src/main/java/com/sv/grupo11/asociacioncomunal/
  ui/          MenuBase (abstracta), MenuAdministrador, MenuSecretario, MenuTesorero,
MenuSocio, MenuFactory
  controller/  AuthController, SocioController, CuotaController, SistemaController
  service/     AuthService, SocioService, CuotaService, ReporteService
  concurrent/  ProcesoCierreMensual, GeneracionCuotasTask, AuditoriaMorosidadTask
  repository/  SocioRepository, CuotaRepository, UsuarioRepository   (interfaces)
  dao/         JdbcSupport (abstracta, generica), SocioDAO, CuotaDAO, UsuarioDAO
  model/       Persona (abstracta), Socio, MiembroDirectiva, Cuota, Usuario, enums
  dto/         SocioMorosoDTO, ResumenCierreDTO
  exception/   AsociacionException + Validacion, ReglaNegocio, RecursoNoEncontrado,
Persistencia, RegistroDuplicado, Conexion
  config/      ConfiguracionBD, DataSourceConfig
  util/        PasswordUtil, DatosDemo
  main/        Main (unico lugar que conoce las implementaciones concretas)
src/main/resources/db.properties   ·   src/test/java/... (55 pruebas)   ·
database/schema.sql, data.sql
```

El Main actúa como raíz de composición: crea el DataSourceConfig, instancia los DAO y los asigna a variables tipadas con las interfaces, construye los servicios y los controladores, y entrega los controladores a los menús. Ninguna otra clase hace new de un DAO.

7. Diagrama UML actualizado

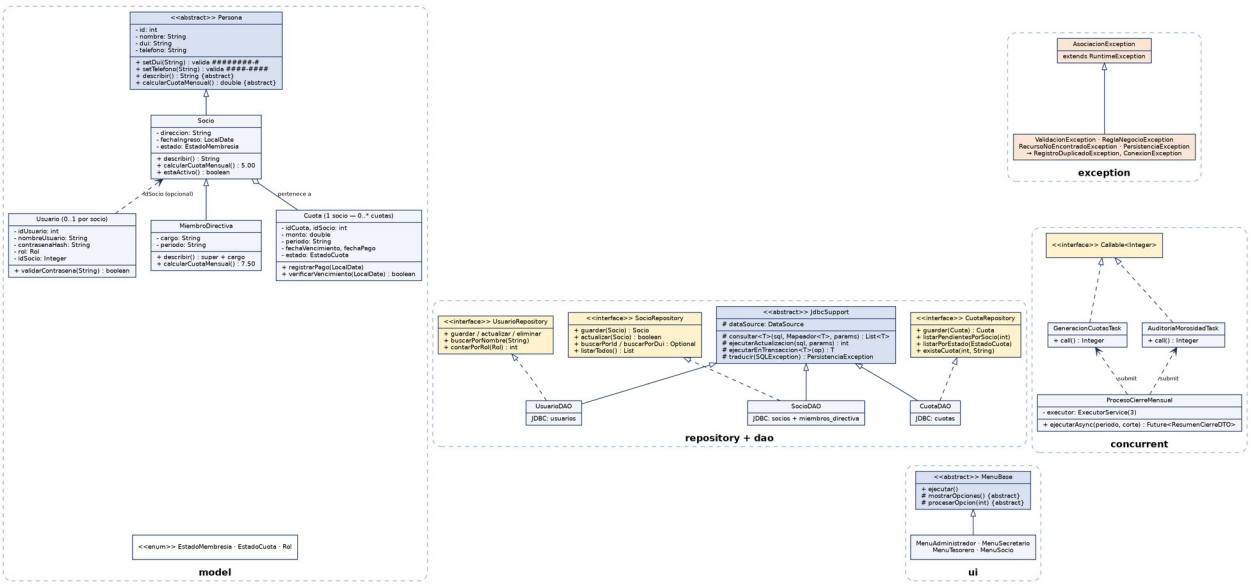


Figura 2. Diagrama de clases por paquete (versión completa en docs/uml.png).

Cambios respecto al Avance 2: ArchivoDAO<T> se sustituye por JdbcSupport (abstracta y genérica) y por las interfaces repository; aparecen los paquetes exception y config; Usuario guarda contrasenaHash en lugar de la contraseña; los ids ya no los calcula la aplicación sino la base de datos.

8. POO aplicada

8.1 Encapsulamiento

Todos los atributos son privados y los setters validan: un objeto nunca queda en estado inválido. La contraseña del usuario no tiene getter de lectura en claro; solo existe validarContrasena(), que compara contra el hash. Los DAO devuelven copias o nuevas instancias, nunca estructuras internas.

```
public void setDui(String dui) {
    if (dui == null || !dui.matches("\\d{8}-\\d"))
        throw new ValidacionException("El DUI debe tener el formato ########-#");
    this.dui = dui;
}
```

8.2 Herencia y especialización

Tres jerarquías: en el modelo, Persona → Socio → MiembroDirectiva (el directivo es un socio que además tiene cargo y periodo, y en la base de datos una fila en miembros_directiva); en acceso a datos, JdbcSupport → SocioDAO, CuotaDAO, UsuarioDAO; en presentación, MenuBase → un menú por rol. Los constructores encadenan con super(...).

8.3 Polimorfismo

calcularCuotaMensual() devuelve $5.00 en Socio y $7.50 en MiembroDirectiva; CuotaService lo invoca sobre una referencia Socio sin preguntar el tipo, y así el cierre mensual genera el monto correcto para cada fila del padrón. describir() del directivo reutiliza la del padre con super.describir(). El Main ejecuta MenuBase.ejecutar() conociendo solo la clase base; MenuFactory decide el menú concreto. El DAO devuelve el mismo tipo que recibe: SocioDAO.mapear() construye un MiembroDirectiva cuando la fila tiene cargo.

```
@Override public double calcularCuotaMensual() {
    return super.calcularCuotaMensual() + APORTE_DIRECTIVA;   // 5.00 + 2.50
}
// CuotaService: el tipo real decide el monto en tiempo de ejecucion
new Cuota(0, socio.getId(), socio.calcularCuotaMensual(), periodo, vencimiento);
```

8.4 Abstracción

Persona, JdbcSupport y MenuBase son clases abstractas: definen lo común y obligan a las subclases a completar lo específico (describir(), obtener el Mapeador de cada entidad, mostrarOpciones()). JdbcSupport concentra en un solo lugar la obtención de conexiones, los parámetros, las transacciones y la traducción de errores; los DAO concretos quedan reducidos a sus sentencias SQL y a su mapeo.

8.5 Interfaces

SocioRepository, CuotaRepository y UsuarioRepository son contratos de acceso a datos: los servicios, el proceso concurrente y los DTO solo conocen estas interfaces. Esto aplica el principio de inversión de dependencias, habilita las pruebas con Mockito (el mock implementa la interfaz) y permitiría cambiar MySQL por otro motor o por archivos sin tocar la lógica de negocio. Además se usan Callable<Integer> para las tareas concurrentes y las interfaces funcionales propias Mapeador<T> y OperacionTransaccional<T>.

9. Colecciones y genéricos

Uso	Dónde	Justificación
ArrayList<T> / List<T>	Resultados de todas las consultas (JdbcSupport.consultar)	Orden de inserción y recorrido secuencial; el ResultSet se vuelca a una lista desacoplada de la conexión, que se cierra de inmediato
List<Socio> polimórfica	GeneracionCuotasTask, listarSociosUI	Socios y directivos conviven en la misma lista; cada elemento calcula su propia cuota
HashMap<Integer, List<Cuota>>	ReporteService.reporteMorosos	Agrupa las cuotas vencidas por socio en una pasada O(n) en vez de O(n·m); clave = id del socio
Optional<T>	buscarPorId / buscarPorDui / login	Hace explícita la ausencia de resultado y evita null
Genéricos propios: Mapeador<T>, consultar<T>, ejecutarEnTransaccion<T>, OperacionTransaccional<T>	dao/JdbcSupport	Una sola implementación de consulta y transacción sirve para Socio, Cuota y Usuario con seguridad de tipos en compilación

Uso	Dónde	Justificación
Future<ResumenCierreDTO>, Callable<Integer>	concurrent/	Resultados tipados de las tareas concurrentes

```
protected <T> List<T> consultar(String sql, Mapeador<T> mapeador, Object... parametros) {
    List<T> resultado = new ArrayList<>();
    try (Connection con = abrirConexion(); PreparedStatement ps =
con.prepareStatement(sql)) {
        asignarParametros(ps, parametros);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) resultado.add(mapeador.mapear(rs));
        }
    } catch (SQLException e) { throw traducir("consultar", e); }
    return resultado;
}
```

10. Programación concurrente

- **Qué proceso se ejecuta concurrentemente:** el cierre mensual. GeneracionCuotasTask inserta en la tabla cuotas la cuota del periodo de cada socio activo; AuditoriaMorosidadTask lee las cuotas PENDIENTES y actualiza a VENCIDA las que superaron su fecha límite.
- **Por qué:** son dos recorridos independientes sobre filas distintas (INSERT de cuotas nuevas vs UPDATE de cuotas viejas) y en un padrón grande tardan; ejecutarlos en paralelo reduce el tiempo total y lanzarlos en segundo plano deja al tesorero seguir registrando pagos. La concurrencia responde a una operación real del dominio, no a una demostración.
- **Mecanismo:** ExecutorService de tamaño fijo 3 (un hilo orquestador + dos tareas Callable<Integer>). ProcesoCierreMensual.ejecutarAsync() devuelve un Future<ResumenCierreDTO>; la UI consulta isDone() y muestra el resumen. Los hilos son daemon y el executor se apaga en el finally del Main.
- **Interacción con la base de datos:** cada tarea trabaja a través de CuotaService y CuotaRepository; cada operación obtiene una conexión del pool (DataSourceConfig), la usa dentro de try-with-resources y la devuelve. Nunca se comparte una Connection entre hilos.
- **Control de problemas de concurrencia:** (1) los ids los asigna MySQL con AUTO_INCREMENT y RETURN_GENERATED_KEYS, eliminando la condición de carrera del antiguo MAX(id)+1; (2) la restricción UNIQUE (id_socio, periodo) rechaza cuotas duplicadas aunque dos hilos inserten a la vez, y la tarea trata ese caso (RegistroDuplicadoException) como omitido; (3) cada escritura corre en su propia transacción con rollback; (4) los DAO no tienen estado compartido ni synchronized, por lo que no serializan al pool; (5) el pool tiene maxActive = 5, suficiente para 3 hilos más la UI.
- **Manejo de excepciones en los hilos:** una PersistenciaException dentro de una tarea no se oculta: se propaga al Future y estadoCierreUI() la muestra desde ExecutionException.getCause().

```
public Future<ResumenCierreDTO> ejecutarAsync(String periodo, LocalDate fechaCorte) {
    return executor.submit(() -> {
        Future<Integer> generadas = executor.submit(new
GeneracionCuotasTask(socios.listarTodos(), periodo, cuotaService));
        Future<Integer> vencidas  = executor.submit(new AuditoriaMorosidadTask(cuotas,
fechaCorte));
        return new ResumenCierreDTO(periodo, generadas.get(), vencidas.get(), duracion);
```

```
    });
}
```

Traza esperada en consola (dos hilos distintos, cierre-2 y cierre-3, trabajando a la vez sobre la base de datos):

```
  [cierre-2] Generacion de cuotas 2026-10 iniciada (5 socios)
  [cierre-3] Auditoria de morosidad iniciada (corte 2026-10-31)
  [cierre-3] Auditoria finalizada: 2 cuota(s) vencida(s)
  [cierre-2] Generacion finalizada: 5 cuota(s), 0 omitida(s)
  [cierre] Cierre 2026-10: 5 cuota(s) generada(s), 2 cuota(s) marcada(s) como vencida(s)
en 118 ms
```

11. JDBC

Elemento	Implementación
Conexión	DataSourceConfig crea el pool con la URL, usuario y clave de db.properties (o de las variables de entorno DB_URL, DB_USER, DB_PASSWORD). verificarConexion() prueba una conexión al arrancar y, si falla, el sistema muestra la causa y no inicia (ConexionException)
DataSource / pool	org.apache.tomcat.jdbc.pool.DataSource; los DAO reciben javax.sql.DataSource por constructor y llaman getConnection(); close() devuelve la conexión al pool
DAO / Repository	Interfaces en repository/ e implementaciones JDBC en dao/, con JdbcSupport como base común
PreparedStatement	Toda sentencia usa parámetros ?; asignarParametros() convierte LocalDate → java.sql.Date, enums → texto y null → setNull. Evita inyección SQL
ResultSet	Cada DAO define un Mapeador<T> (método mapear) que convierte la fila en entidad; SocioDAO decide Socio o MiembroDirectiva según la columna cargo
executeQuery()	consultar() y consultarEntero() (SELECT, COUNT)
executeUpdate()	ejecutarActualizacion() (UPDATE, DELETE) e insertarYObtenerId() (INSERT con RETURN_GENERATED_KEYS)
Inserción	SocioDAO.guardar (socios + miembros_directiva en una transacción), CuotaDAO.guardar, UsuarioDAO.guardar
Consulta	buscarPorId, buscarPorDui, buscarPorNombre, listarTodos, listarPorSocio, listarPendientesPorSocio, listarPorEstado, existeCuota, contarPorRol
Actualización / eliminación	actualizar en los tres DAO; UsuarioDAO.eliminar (DELETE)
Transacciones	ejecutarEnTransaccion(): setAutoCommit(false), commit, rollback en catch y restauración de autocommit + close en finally
Manejo de errores	traducir(): SQLIntegrityConstraintViolationException → RegistroDuplicadoException; resto → PersistenciaException con la causa original. Fallo al obtener conexión → ConexionException
Cierre de recursos	try-with-resources en Connection, PreparedStatement y ResultSet en todos los métodos

```
@Override public Socio guardar(Socio socio) {
    int idGenerado = ejecutarEnTransaccion(con -> {
        int id = insertarYObtenerId(con, "INSERT INTO socios (nombre, dui, telefono,
direccion, fecha_ingreso, estado) VALUES (?,?,?,?,?,?)",
                socio.getNombre(), socio.getDui(), socio.getTelefono(),
socio.getDireccion(), socio.getFechaIngreso(), socio.getEstado());
        if (socio instanceof MiembroDirectiva) {
            MiembroDirectiva m = (MiembroDirectiva) socio;
            insertarYObtenerId(con, "INSERT INTO miembros_directiva (id_socio, cargo,
periodo) VALUES (?,?,?)", id, m.getCargo().toUpperCase(), m.getPeriodo());
        }
        return id;
    });
    return buscarPorId(idGenerado).orElseThrow(...);
}
```

12. Configuración del DataSource

Abrir una conexión a MySQL cuesta decenas de milisegundos (handshake TCP y autenticación). El pool mantiene conexiones abiertas y las presta; el close() de un DAO no cierra el socket, devuelve la conexión al pool. Valores configurados en db.properties y aplicados en DataSourceConfig:

Parámetro	Valor	Propósito y justificación para esta aplicación
validationQuery	SELECT 1	Consulta mínima para comprobar que la conexión sigue viva antes de entregarla
initialSize	2	Conexiones creadas al arrancar: una para la UI y una de reserva; evita abrir conexiones durante el login
minIdle	2	El pool nunca baja de 2 inactivas, así una operación nueva no espera a que se abra una conexión
maxIdle	5	Tras el cierre mensual (3 hilos) las conexiones extra se conservan hasta la evicción
maxActive	5	Máximo simultáneo: 3 hilos del cierre + UI + 1 de margen. Más sería desperdicio en una app de consola
maxWait	5000 ms	Si el pool está lleno, el hilo espera a lo sumo 5 s y falla con excepción clara en vez de colgarse
testOnBorrow	true	Valida la conexión al prestarla: una conexión cerrada por MySQL nunca llega a un DAO
testOnReturn	false	Validar al devolver duplicaría el trabajo de testOnBorrow
testWhileIdle	true	El hilo de evicción valida las conexiones inactivas y descarta las muertas
testOnConnect	true	Valida cada conexión recién creada antes de agregarla al pool
timeBetweenEvictionRunsMillis	30000	Cada 30 s el pool revisa las conexiones inactivas
minEvictableIdleTimeMillis	60000	Una conexión inactiva más de 1 min se cierra (respetando minIdle); MySQL cierra por inactividad a las 8 h, así que 60 s es seguro

Parámetro	Valor	Propósito y justificación para esta aplicación
removeAbandoned	true	Si un DAO olvidara devolver una conexión (fuga), el pool la recupera
removeAbandonedTimeout	60 s	Tiempo tras el cual una conexión prestada y no devuelta se considera abandonada; ninguna operación del sistema dura más de unos segundos
validationInterval	30000 ms	No se vuelve a validar una conexión probada hace menos de 30 s (evita un SELECT 1 por cada consulta)

El menú de Administrador incluye la opción 'Estado del pool de conexiones', que muestra conexiones activas, inactivas y total en tiempo real, útil para evidenciar que las conexiones se reutilizan y que el cierre mensual toma varias a la vez.

13. Cohesión y acoplamiento

- **Clase con buena cohesión — CuotaService:** todos sus métodos tratan una sola cosa, el ciclo de vida de una cuota (generar, registrar pago, listar pendientes e historial). No sabe de consola, de SQL ni de hilos; recibe sus dependencias por constructor y delega la persistencia al repositorio.
- **Dependencia reducida:** en el Avance 2 los servicios dependían de las clases concretas SocioDAO, CuotaDAO y UsuarioDAO, y en la primera migración a JDBC los DAO dependían de la clase estática ConexionBD con la contraseña escrita en el código. Ahora los servicios dependen de las interfaces repository y los DAO reciben un javax.sql.DataSource por constructor. Ninguna clase, salvo el Main, conoce una implementación concreta.
- **Decisión de diseño que disminuye el acoplamiento:** inyección de dependencias por constructor con el Main como raíz de composición, más las interfaces repository. El efecto medible es que las 55 pruebas unitarias corren sin MySQL: Mockito sustituye los repositorios. Un segundo ejemplo es MenuFactory, que mantiene al Main independiente de los cuatro menús concretos.

14. Legibilidad y reutilización

Revisión realizada sobre el código del Avance 2 y de la primera migración a JDBC. Ejemplos antes/después:

Antes	Después	Mejora
ConexionBD con DriverManager, usuario y contraseña como constantes en el código fuente y un main() de prueba dentro del DAO	config/ConfiguracionBD + DataSourceConfig; credenciales en db.properties o variables de entorno; pool de conexiones	Seguridad, configuración externa, reutilización de conexiones, responsabilidad única
consultarEntero(), consultar(), asignarFecha() y el catch de SQLException copiados en los tres DAO (≈ 90 líneas repetidas)	JdbcSupport abstracta y genérica; cada DAO solo tiene SQL y su mapear()	Eliminación de código duplicado; los DAO pasaron de ~190 a ~100 líneas
siguienteId() calculaba MAX(id)+1 en la aplicación	AUTO_INCREMENT + RETURN_GENERATED_KEYS; guardar() devuelve la entidad con su id	Elimina una condición de carrera entre hilos

Antes	Después	Mejora
throw new IllegalStateException("No se pudo guardar...") en todas partes; la UI capturaba IllegalArgumentException | IllegalStateException	Jerarquía propia en exception/; la UI captura AsociacionException	Errores con significado (validación, negocio, persistencia, duplicado, conexión) y un solo punto de manejo
ArchivoDAO<T> seguía en el proyecto sin ser usado tras migrar a JDBC	Eliminada; su rol lo cumple JdbcSupport	Código muerto fuera
Usuario.getContrasena() devolvía el campo que en realidad contendría el hash	getContrasenaHash(); PasswordUtil.hash() y verificar()	El nombre dice la verdad; contraseñas protegidas
Métodos synchronized en los DAO (heredados de los archivos .dat)	DAO sin estado; la consistencia la dan el pool, las transacciones y las restricciones de la BD	No se serializa el acceso a la base; el pool se aprovecha
Fuentes bajo src/ y un .jar copiado a mano en lib/	Layout Maven estándar (src/main, src/test, pom.xml) con dependencias declaradas	Organización de paquetes reconocible y construcción reproducible

15. Manejo de excepciones

Jerarquía propia en el paquete exception: AsociacionException (base, extiende RuntimeException) y seis especializaciones: ValidacionException (datos de entrada), ReglaNegocioException (cuota ya pagada, DUI repetido, último administrador), RecursoNoEncontradoException, PersistenciaException (errores JDBC, envuelve la SQLException original) y sus hijas RegistroDuplicadoException (restricción UNIQUE) y ConexionException (no se obtuvo conexión).

Elemento	Dónde se usa
try / catch	JdbcSupport en cada operación JDBC; MenuBase.ejecutar() con un único catch (AsociacionException) para informar y continuar; Main al verificar la conexión
finally	JdbcSupport.ejecutarEnTransaccion(): restaura autocommit y devuelve la conexión al pool pase lo que pase; Main: apaga el executor y cierra el pool al terminar
throw	Setters de Persona (ValidacionException), Cuota.registrarPago (ReglaNegocioException), servicios (RecursoNoEncontradoException, ReglaNegocioException), traducir() en JdbcSupport
throws	Mapeador.mapear(ResultSet) throws SQLException; OperacionTransaccional.ejecutar(Connection) throws SQLException; insertarYObtenerId(...) throws SQLException: el error JDBC sube hasta el punto donde se traduce
Excepciones JDBC	SQLException se captura en un solo lugar (JdbcSupport) y se traduce a la excepción propia conservando la causa; la UI nunca ve java.sql
Errores de negocio	ReglaNegocioException desde los servicios y el modelo; en las tareas concurrentes se distingue del error técnico: un duplicado se omite, una falla de BD se propaga al Future

No se usa catch (Exception e) ni catch (Throwable t). El catch de ExecutionException en estadoCierreUI() existe porque es la forma en que Future entrega la excepción lanzada dentro del hilo, y se muestra su getCause().

16. Pruebas unitarias

Herramientas: JUnit 5.10 (Jupiter) y Mockito 5.11 con MockitoExtension. Las dependencias externas se aíslan con @Mock: los servicios se prueban contra repositorios simulados y los controladores contra servicios simulados, por lo que las pruebas no requieren MySQL. Ejecución: mvn test (55 pruebas, 12 clases).

Clase de prueba	Qué verifica	Pruebas
model/PersonaTest	Validaciones de DUI, teléfono y nombre; el setter conserva el valor anterior ante un dato inválido	5
model/MiembroDirectivaTest	Polimorfismo: $5.00 vs $7.50; describir() reutiliza al padre; un directivo es un Socio	3
model/CuotaTest	Estado inicial, monto inválido, registrarPago, pago doble (error), verificarVencimiento	5
util/PasswordUtilTest	Formato del hash, salt aleatorio, verificación correcta/incorrecta, compatibilidad legada, contraseña vacía	5
service/SocioServiceTest	Registro exitoso (ArgumentCaptor), DUI duplicado (never guardar), datos inválidos (verifyNoInteractions), no encontrado, actualizar, cambiar estado	6
service/CuotaServiceTest	Monto por tipo de socio, vencimiento a fin de mes, cuota duplicada, pago exitoso, cuota inexistente, pago doble, socio inexistente	7
service/ReporteServiceTest	Agrupación por socio, suma de montos y orden; lista vacía	2
service/AuthServiceTest	Login correcto/incorrecto/inexistente, hash al crear usuario, contraseña corta, SOCIO sin id, nombre repetido, último administrador, eliminar	10
controller/CuotaControllerTest · SocioControllerTest	Delegación al servicio correcto con los argumentos recibidos	6
concurrent/ GeneracionCuotasTaskTest	Solo socios activos; omite duplicados (ReglaNegocio y RegistroDuplicado); lista vacía	2
dao/JdbcSupportTest	Conversión de parámetros sobre un PreparedStatement simulado; traducción de SQLException; DataSource obligatorio	4

Aserciones empleadas: assertEquals (con delta para double), assertTrue/False, assertNull/NotNull, assertSame, assertThrows (capturando la excepción para inspeccionar su mensaje), assertAll; verificaciones de Mockito: verify(times/never), verifyNoInteractions y ArgumentCaptor para inspeccionar el objeto que llegó al repositorio.

```
@Test @DisplayName("Polimorfismo: la cuota generada a un miembro de directiva es de
$7.50")
```

```
void generarCuota_directiva_montoDiferenciado() {
    Socio directiva = new MiembroDirectiva(4, "Carlos", "02345678-1", "7777-1010", "x",
LocalDate.now(), "Tesorero", "2026-2028");
    when(cuotaRepository.existeCuota(4, "2026-10")).thenReturn(false);
    when(cuotaRepository.guardar(any(Cuota.class))).thenReturn(new Cuota(15, 4, 7.50,
"2026-10", LocalDate.of(2026, 10, 31)));
    service.generarCuota(directiva, "2026-10");
    ArgumentCaptor<Cuota> captor = ArgumentCaptor.forClass(Cuota.class);
    verify(cuotaRepository).guardar(captor.capture());
    assertEquals(7.50, captor.getValue().getMonto(), 0.001);
}
```

16.1 Evidencia de ejecución

Las pruebas del modelo y de utilidades (18) se ejecutaron durante el desarrollo con resultado 18/18; el reporte completo de las 55 se genera con mvn test y queda en target/surefire-reports. (Insertar aquí la captura de la salida de mvn test: Tests run: 55, Failures: 0, Errors: 0.)

```
  PASS PersonaTest.duiInvalido_lanzaExcepcion
  PASS MiembroDirectivaTest.directiva_cuotaDiferenciada
  PASS CuotaTest.registrarPagoDosVeces_lanzaReglaNegocio
  PASS PasswordUtilTest.hash_usaSaltAleatorio
  ...
== 18 pasaron, 0 fallaron ==
```

17. Git / GitHub

Repositorio público: https://github.com/isabelsaraviai689-max/Asociacion-Comunal. El proyecto se versiona desde el Avance 2 con commits por integrante, ramas por funcionalidad y Pull Requests revisados antes de integrar a main; el documento de cada avance se versiona en la raíz (ahora en docs/).

Integrante	Aporte versionado en esta entrega	Mensaje de commit
Isabel Ramos	Migración inicial a JDBC/MySQL, esquema SQL, README, documento final	feat: agrega conexion JDBC y DAO sobre MySQL · docs: documento final
Luis Alonso Alvarenga	DataSourceConfig y pool, JdbcSupport, interfaces repository, paquete exception, concurrencia sobre BD, pruebas	feat: configura DataSource y pool de conexiones · refactor: extrae JdbcSupport e interfaces repository · test: agrega pruebas de servicios y controladores
Camila Salvador	database/schema.sql y data.sql, modelo y mapeo Socio/MiembroDirectiva	feat: esquema y datos de la base asociacion_comunal
Esmeralda López	Historias de usuario HU-011/HU-012, controladores, menús (eliminar usuario, estado del pool)	feat: agrega eliminacion de usuarios y estado del pool al menu
Diego Escobar	Ejecución de pruebas, evidencias, pruebas funcionales por rol	test: evidencias de ejecucion de pruebas y pruebas funcionales

18. Conclusiones

La arquitectura por capas decidida en el Avance 2 demostró su valor en esta entrega: la persistencia cambió por completo, de archivos serializados a MySQL con un pool de conexiones, y los servicios, controladores y menús no se modificaron. Introducir las interfaces repository terminó de aislar la lógica de negocio del mecanismo de almacenamiento, y esa misma decisión fue la que hizo posible escribir 55 pruebas unitarias que corren sin base de datos.

Migrar a JDBC obligó a repensar la concurrencia. Lo que en el Avance 2 se resolvía con métodos synchronized sobre una lista en memoria, en una base de datos se resuelve mejor delegando: ids generados por el motor, restricciones UNIQUE que rechazan duplicados aunque dos hilos inserten a la vez, transacciones por operación y un pool dimensionado para los hilos del cierre mensual. El resultado es un proceso concurrente que interactúa con la base de datos de forma segura y cuyos errores no quedan ocultos.

La jerarquía de excepciones propia y la revisión de legibilidad produjeron un código más honesto: los errores tienen nombre y causa, el código duplicado de los DAO se concentró en una clase abstracta genérica, las contraseñas dejaron de guardarse en claro y el proyecto adoptó una estructura Maven estándar. El equipo, reorganizado a mitad de ciclo, cerró el proyecto cumpliendo o justificando cada compromiso del avance anterior.

Bibliografía

- Oracle. (2024). The Java Tutorials: JDBC Database Access; Concurrency (Executors, Callable and Future). https://docs.oracle.com/javase/tutorial/
- Apache Software Foundation. (2024). The Tomcat JDBC Connection Pool — Attributes. https://tomcat.apache.org/tomcat-10.1-doc/jdbc-pool.html
- JUnit Team. (2024). JUnit 5 User Guide. https://junit.org/junit5/docs/current/user-guide/
- Mockito. (2024). Mockito framework site and documentation. https://site.mockito.org/
- Goetz, B., et al. (2006). Java Concurrency in Practice. Addison-Wesley.
- Martin, R. C. (2017). Clean Architecture. Prentice Hall.
- Bloch, J. (2018). Effective Java (3.ª ed.). Addison-Wesley.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
- NIST. (2023). SP 800-132: Recommendation for Password-Based Key Derivation. https://csrc.nist.gov/